AI APIs as Data Tools

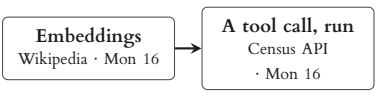
Week 13 · APIs module · Textbook Chapter 13

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

November 9, 13 & 16, 2026

This week at a glance





Monday, Nov 9 | Part 1: a PDF becomes Markdown
Wednesday, Nov 11 | No class **Friday, Nov 13** | Part 2: a model returns JSON Then 15 minutes on portfolio checkpoint 3.
Monday, Nov 16 | Part 3: embeddings, and one tool call

Keys

`DATALAB_KEY` and `OPENROUTER_KEY`. Part 3 also uses week 11's `CENSUS_KEY`.

Due

Sunday, Nov 15: portfolio checkpoint 3.
Sunday, Nov 22: chapter 13's Recommended Exercises.

1. Monday · Live-Coding Lab

This week

Find it | `labs/week-13` in the course repository

1. Open `lab-13.ipynb` and click **Download raw file**.
2. Open it in JupyterLab, in the `webdata` environment.

Use it | one part a day

- I type the code on screen. You type it in your copy.
- Each part starts at its own setup step.

Fall behind?

Copy the cell from `lab-13-key.ipynb`, beside it, and keep going.

Errors

A `429` means a free model is busy: wait, and send it again.

A model called like a function

A chatbot answers a person. This week a model answers a program.

We send a request and get data back, as with week 11's APIs. Then we check that data against something we trust.

Who sees what you send

Every page goes to Datalab, or to OpenRouter and the model's owner. Send nothing you couldn't publish.

Step 0: store two keys

Get a key at datalab.to and at openrouter.ai.

Store both in a terminal. Then close it, open a new one, and start Jupyter from there.

```
conda env config vars set DATALAB_KEY="..." -n webdata
conda env config vars set OPENROUTER_KEY="..." -n webdata
```

Step 0: check them

Print whether each key is set. Never print the key itself.

Does yours say `MISSING`? Run the given `getpass()` cell under it.

```
print("DATALAB_KEY:", "set"
      if os.environ.get("DATALAB_KEY") else "MISSING")
print("OPENROUTER_KEY:", "set"
      if os.environ.get("OPENROUTER_KEY") else "MISSING")
```

Step 1: submit a document

We send Datalab a link to week 9's 2024 report. Datalab fetches the PDF and starts reading it.

What comes back first is a ticket. Which field is it?

Docs: Datalab's Convert API

```
data = {"file_url": book + "revenue-report-2024-12.pdf",
        "output_format": "markdown",
        "mode": "balanced", "paginate": "true",
        "skip_cache": "true"}
job = requests.post(f"{datalab_base}/convert", data=data,
    headers={"X-API-Key": datalab_key, **HEADERS}).json()
print(job)
```

Step 2: ask about the ticket

The ticket, `request_check_url`, is where the result will appear. Ask it once.

`processing` means Datalab is still reading.

```
check = requests.get(job["request_check_url"],
    headers={"X-API-Key": datalab_key, **HEADERS}).json()
print(check["status"])
```

Step 2: wait for the result

This cell is given. It asks again every two seconds until the job is done. Then it prints the pages, the cost, and the time.

Discuss: what would this loop do if Datalab never finished?

```
while check["status"] == "processing":
    time.sleep(2)
    check = requests.get(job["request_check_url"],
        headers={"X-API-Key": datalab_key,
                 **HEADERS}).json()
print("status:", check["status"])
print(check["page_count"], "pages,",
      check["total_cost"], "cents")
```

Step 3: split the Markdown into pages

Each page starts with a marker such as `{14}` and a line of dashes. Split on the markers, and drop the empty piece before the first one.

Never print `pages[0]`: the cover gives a staff member's phone and email.

```
markdown = check["markdown"]
pages = re.split(r"\n*\{\d+\}-{10,}\n*",
                 markdown)[1:]
print(len(pages), "pages")
print(pages[1][:1200])
```

Step 3: a picture of a table

Page 15, Exhibit 3, is an image. Week 9's pdfplumber found no text in it.

Print the first lines of the Markdown table Datalab wrote. Does it look like the PDF?

```
exhibit = pages[14][pages[14].index("|"):]
for line in exhibit.splitlines()[:10]:
    print(line[:150])
```

What Datalab read, and what its model wrote

Datalab **read** Chart 8's figures from its slices. Its model also **wrote** a sentence about them.

The sentence calls $1,262,688 the largest slice. Is it?

Discuss: how would you tell what was read from what was written?

Chart 8, page 9	Amount
State Share-Back	$529,220
Medical Marijuana (3.86% Base)	$52,937
Recreational Marijuana (3.86% Base)	$1,262,688
Recreational Marijuana (3.5% + 5.0% Excise)	$1,341,007

Step 4: split each row into cells

Week 9's first check: a row's twelve months should add up to its total.

First, split each line of the table on `|`, and strip the bold tags and commas. A row we want starts with a year.

```
for line in pages[14].splitlines():
    cells = [c.strip().replace("<b>", "")
             .replace("</b>", "").replace(",", "")
             for c in line.split("|")]
    if len(cells) > 16 and re.fullmatch(r"20\d\d",
                                        cells[2]):
        ...
```

Step 4: add up the months

Add the twelve months, and compare the sum with the row's total. Print a row only if it misses by more than $5.

Nothing printed? What can a sum still miss?

```
checked = 0
for line in pages[14].splitlines():
    ...
        months = sum(int(c) for c in cells[3:15])
        checked += 1
        if abs(months - int(cells[15])) > 5:
            print(cells[2], months, cells[15])
print(checked, "rows checked")
```

2. Wednesday

I was so very very sick



3. Friday · Live-Coding Lab

Today: Part 2, a model as a function

We will use OpenRouter to call a free language model through an API. We send an instruction and one page of a report, and get back JSON.

- Find the page with the summary table's last row
- Ask for its totals as JSON
- This requires us to define the fields we want and their types
- Check them against `pdfplumber`'s figures; then do all seven reports

Start at Step 6

Download the new `lab-13.ipynb`. Today needs only `OPENROUTER_KEY`.

Free models are shared

The whole class uses the same free models. A busy one answers `429`.

Step 6: set up again

Put your email in `HEADERS`, and check the key.

Today we read the book's saved Markdown of all seven reports, so no one pays Datalab again.

```
print("OPENROUTER_KEY:", "set"
      if os.environ.get("OPENROUTER_KEY") else "MISSING")
openrouter_base = "https://openrouter.ai/api/v1"
saved = ("https://raw.githubusercontent.com/cuinfoscience/"
         "Web-Data-Science-Book/main/data/ch-13/")
```

Step 7: find the summary page

The model sees only what we send. We send one page: the one with the row *Total Sales and Use Tax*.

Read the start of that page. Where is the row we want?

```
report = saved + "revenue-report-2024-12.md"
markdown = requests.get(report, headers=HEADERS).text
pages = re.split(r"\n*\{\d+\}-{10,}\n*", markdown)[1:]
summary = [page for page in pages
           if "Total Sales and Use Tax" in page][0]
print(summary[:1000])
```

Step 8: the shape of the answer

A **JSON Schema** names the fields we want and the type of each.

This cell is given. In the notebook, every field also has a description, and `MONTHS` lists the twelve months.

```
SCHEMA = {"type": "object",
    "properties": {
        "month": {"type": "string", "enum": MONTHS},
        "year": {"type": "integer"},
        "total_prior": {"type": "integer"},
        "total_current": {"type": "integer"}},
    "required": ["month", "year",
                 "total_prior", "total_current"]}
```

Step 8: the instruction

The instruction tells the model its job. It is also given.

Which words in it try to stop the model from guessing?

```
INSTRUCTION = (
    "You extract figures from one page of a City of "
    "Boulder revenue report. Copy numbers exactly as "
    "printed, without $ or commas. Use only the page.")
```

Step 8: the request's body

`models`: three free models. If the first is busy, OpenRouter tries the next.

`temperature` 0 asks for the model's most likely answer.

The next two frames fill in the . . . .

```
MODELS = ["nvidia/nemotron-3.5-super-vl:free",
          "google/gemma-4-31b-it:free",
          "dots-studio/dots-3-note-preview:free"]
body = {"models": MODELS, "temperature": 0,
        "messages": [...],
        "response_format": {...},
        "provider": {"require_parameters": True}}
```

Step 8: what the model reads

Two messages. The `system` message is our instruction. The `user` message is the page.

There is no conversation: one request, one answer.

```
"messages": [
    {"role": "system", "content": INSTRUCTION},
    {"role": "user", "content": summary}],
```

Step 8: what the model must return

`response_format` asks for JSON in `SCHEMA`'s shape. `strict`: no other fields.

`require_parameters` sends the request only to models that accept this.

Docs: OpenRouter's structured outputs

```
"response_format": {
    "type": "json_schema",
    "json_schema": {"name": "totals", "strict": True,
                    "schema": SCHEMA}},
"provider": {"require_parameters": True}
```

Step 8: send it

Post the body with your key in the header.

Print three things: any error, which model answered, and `usage`, the tokens in and out. Did the first model answer?

```
response = requests.post(
    f"{openrouter_base}/chat/completions", json=body,
    headers={"Authorization": f"Bearer {openrouter_key}",
             **HEADERS}).json()
print(response.get("error"))
print(response["model"], response["model"] in MODELS)
print(response["usage"])
```

Step 8: read the answer

The answer arrives as text that holds JSON. `json.loads()` turns it into a dictionary.

Look at your four fields. Which would you trust without checking?

```
answer = json.loads(
    response["choices"][0]["message"]["content"])
print(answer)
```

Step 9: check it against week 9

`TRUTH`, given, holds week 9's pdfplumber figures for each report. That is our **ground truth**.

Do both totals match? Is the year 2024?

```
prior, current = TRUTH["revenue-report-2024-12"]
print("prior: ", answer["total_prior"], prior,
      answer["total_prior"] == prior)
print("current:", answer["total_current"], current,
      answer["total_current"] == current)
print(answer["year"], answer["month"])
```

Microsprint: the same request, a different model

Send the same body to the third model alone.

Does it return the same four fields? Does your neighbor's run agree with yours?

```
response = requests.post(
    f"{openrouter_base}/chat/completions",
    json={**body, "models": [MODELS[2]]},
    headers={"Authorization": f"Bearer {openrouter_key}",
             **HEADERS}).json()
print(response["model"], json.loads(
    response["choices"][0]["message"]["content"]))
```

Same request, different answers

Run Step 8 twice, and the answer can change. We have seen the year, and once both totals, change between runs.

`temperature` 0 asks for the most likely answer. The model still runs on shared machines, and small differences in the arithmetic can change which answer comes out.

So record

the model that answered, the date, and the raw response.

Step 10: Step 8 as a function

We will send Step 8's request seven times, once for each report. Only the page changes.

So put the request in a function that takes the page.

```
def extract_totals(page):
    body = {"models": MODELS, "temperature": 0,
            "messages": [..., {"role": "user",
                               "content": page}], ...}
    response = requests.post(
        f"{openrouter_base}/chat/completions",
        json=body, headers=...).json()
    content = response["choices"][0]["message"]["content"]
    return json.loads(content), response["model"]
```

Step 10: seven reports

For each report, find its summary page and send it.

`time.sleep(2)` spaces out the requests to a shared free model.

```
rows = []
for report, (prior, current) in TRUTH.items():
    md = requests.get(saved + report + ".md",
                      headers=HEADERS).text
    pages = re.split(r"\n*\{\d+\}-{10,}\n*", md)[1:]
    summary = [page for page in pages
               if "Total Sales and Use Tax" in page][0]
    answer, model = extract_totals(summary)
```

Step 10: record each answer, and check it

Each row keeps the model's answer and whether each total matches week 9.

Which reports came back wrong in your table? Are they the ones your neighbor got wrong?

```
    rows.append({"report": report, "model": model,
        **answer,
        "prior_ok": answer["total_prior"] == prior,
        "current_ok": answer["total_current"] == current})
    time.sleep(2)
results = pd.DataFrame(rows)
print(results.drop(columns="model").to_string())
```

One way it goes wrong

The 2019 summary ends with two total rows. The last one is the year's total.

If your 2019 is wrong, compare its numbers with these rows. Which did the model copy?

	2018	2019
Total … current economic activity	$139,292,879	$139,548,151
Total Sales and Use Tax	$140,609,751	$141,954,650

Step 10: the schema, in the instruction

Add the schema to the instruction, as text. A given cell runs the loop again.

Compare the two tables. Was the problem the model, or what we sent it?

```
INSTRUCTION = (INSTRUCTION
    + " Answer with JSON that fits this schema: "
    + json.dumps(SCHEMA))
```

Friday's last 15 minutes: portfolio checkpoint 3

Due Sunday, Nov 15 | Analyze your data

- **An analysis notebook** that runs top to bottom, with its output visible
- **Results that answer at least part of your question**: the counts or comparisons your proposal named
- **At least one figure** saved in your repository, with a title, labeled axes, and a one-sentence caption
- **The README's Methods section** and your preliminary **Findings**

Run your analysis notebook once, top to bottom, here, where you can ask. Used a model anywhere? Say in Methods which one, when, and what you checked its output against.

4. Monday, Nov 16 · Live-Coding Lab

Today: Part 3, search by meaning, and tools

On Friday the model read one page that we chose. With a hundred passages, how do we find the ones worth reading?

An **embedding** turns a passage into a list of numbers. Passages about the same thing get similar numbers.

Start at Step 11

Download the new `lab-13.ipynb`. Today uses `OPENROUTER_KEY` and week 11's `CENSUS_KEY`.

Five articles

Obama, Hillary Clinton, Trump, Biden and Harris, on January 1 of each year, 2004–2026.

Step 11: set up

Step 11's first two cells are given. Put your email in `HEADERS`, then run both.

Both keys should say `set`. If one says `MISSING`, the second cell asks for it.

The lead

The paragraphs above an article's first heading. Its job is to summarize the article.

Step 11: find one old version

Which revision of Obama's article did a reader see on January 1, 2009? Start at that moment, walk back in time, and stop after one.

Docs: the Action API's revisions

```
params = {"action": "query", "titles": "Barack Obama",
          "prop": "revisions", "rvprop": "ids|timestamp",
          "rvlimit": 1, "rvstart": "2009-01-01T00:00:00Z",
          "rvdir": "older", "format": "json"}
data = requests.get(API, params=params,
                    headers=HEADERS).json()
page = list(data["query"]["pages"].values())[0]
revision = page["revisions"][0]
```

Step 11: get its lead as HTML

`action=parse` returns that revision as HTML. `section=0` asks for the lead alone.

Read the paragraphs. Does anything look out of place?

Docs: parse

```
params = {"action": "parse", "oldid": revision["revid"],
          "prop": "text", "section": 0,
          "format": "json", "formatversion": 2}
html = requests.get(API, params=params,
    headers=HEADERS).json()["parse"]["text"]
soup = BeautifulSoup(html, "html.parser")
for paragraph in soup.find_all("p"):
    print(paragraph.get_text(" ", strip=True)[:150])
```

Step 11: clean it

Wikipedia draws an old version with today's templates. Deleted templates come back as stray links.

This given cell deletes them, with the tables, footnote marks and error messages, then joins the paragraphs.

```
for tag in soup.select("table, .reference, .hatnote, "
       ".shortdescription, style, "
       "a.new[title^='Template:'], .error"):
    tag.decompose()
paragraphs = [p.get_text(" ", strip=True)
              for p in soup.find_all("p")
              if p.get_text(strip=True)]
lead = re.sub(r" ([,.;:])", r"\1", "\n\n".join(paragraphs))
```

Step 11: the saved leads

A given cell loads all 116 leads, collected and cleaned this way on November 9.

Is your 2009 lead the same as the saved one?

```
rows = leads[(leads["title"] == "Barack Obama")
             & (leads["year"] == "2009")]
saved_lead = rows["lead"].iloc[0]
print(lead == saved_lead)
```

Step 12: embed one lead

Send one lead to the embeddings endpoint. It comes back as a **vector**, a list of numbers.

How many numbers? No single one means anything. What matters is how two vectors compare.

```
EMBEDDER = "nvidia/nemotron-3-embed-1b:free"
one = requests.post(f"{openrouter_base}/embeddings",
    json={"model": EMBEDDER,
          "input": [leads["lead"][0]]},
    headers={"Authorization": f"Bearer {openrouter_key}",
             **HEADERS}).json()
print(len(one["data"][0]["embedding"]), "numbers")
```

Step 12: embed them all

A given cell embeds every lead, 20 to a request, and scales each vector to length 1.

It uses one model and no fallback list: two models' vectors can't be compared. If the model's daily limit stops you, the next given cell loads the vectors saved on November 9.

Cosine similarity

How nearly two vectors point the same way: 1 if exactly. For vectors of length 1, it is their dot product, `@`.

Step 12: compare each lead with today's

Keep each article's current vector. Then compare every lead with its own article's current lead.

The current rows should print 1. Do they?

```
today = {}
for i, row in leads.iterrows():
    if row["year"] == "current":
        today[row["title"]] = vectors[i]
leads["similarity"] = [vectors[i] @ today[title]
    for i, title in enumerate(leads["title"])]
print(leads[leads["year"] == "current"][
    ["title", "similarity"]])
```

Step 12: a row per year

Pivot to a row per year and a column per article. A given cell plots it.

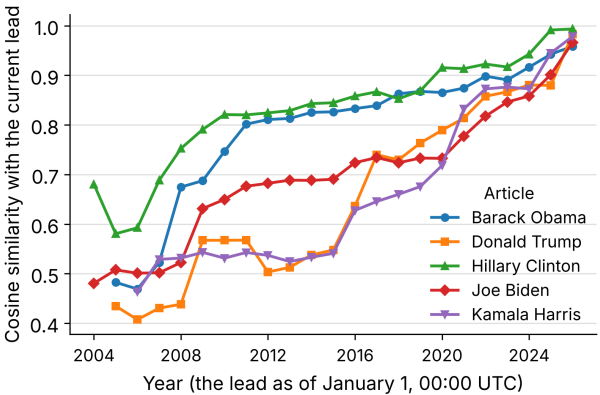
Where do the lines jump? What happened that year?

```
table = leads[leads["year"] != "current"].pivot(
    index="year", columns="title", values="similarity")
table.index = table.index.astype(int)
print(table.round(3))
```

Step 12: twenty-three years of leads

Your plot should look like this one.

A similarity says how far a lead's words moved toward today's. It doesn't say why. For that, read the leads.



Microsprint: the biggest jump

Pick an article. `diff()` gives each year's change, and `idxmax()` the year of the biggest rise.

Read the lead before and after. Was it a new office, or a rewrite?

```
title = "Kamala Harris"
jumps = table[title].diff()
year = jumps.idxmax()
print(year, round(jumps.max(), 3))
for y in [year - 1, year]:
    print(y, leads[(leads["title"] == title)
        & (leads["year"] == str(y))]["lead"].iloc[0][:400])
```

Step 13: the shape of an answer with sources

Now ask a question, and require the model to name its sources.

This schema is given. `year` may be `null`: no passage says it.

```
ANSWER_SCHEMA = {
    "type": "object",
    "properties": {
        "year": {"type": ["integer", "null"]},
        "source_ids": {"type": "array",
                       "items": {"type": "string"}}},
    "required": ["year", "source_ids"],
    "additionalProperties": False}
```

Step 13: search by meaning

Embed the question as Step 12 embedded a lead. Score every lead against it, and keep that article's best three.

This is **retrieval**: find the passages before you ask.

```
def search(question, title):
    response = requests.post(
        f"{openrouter_base}/embeddings", headers=...,
        json={"model": EMBEDDER, "input": [question]}).json()
    query = np.array(response["data"][0]["embedding"])
    query = query / np.linalg.norm(query)
    leads["score"] = vectors @ query
    best = leads[leads["title"] == title]
    return best.nlargest(3, "score")
```

Step 13: find three leads

Which three Biden leads score highest?

Join them into `passages`, each headed by its ID, such as `Joe Biden 2022`.

```
question = ("In which year did the article first say "
            "Joe Biden is the 46th president of the "
            "United States?")
top = search(question, "Joe Biden")
print(top[["id", "score"]])
passages = "\n\n".join(f"{row.id}: {row.lead}"
                       for row in top.itertuples())
```

Step 13: ask

Friday's body, with the new instruction and schema. The user message holds the question and the passages.

Which year did it give, and which sources?

```
body = {"models": MODELS, "temperature": 0,
    "messages": [
        {"role": "system", "content": ANSWER_INSTRUCTION},
        {"role": "user", "content":
            f"Question: {question}\n\n{passages}"}],
    "response_format": {...}, "provider": {...}}
response = requests.post(...).json()
print(response["model"], json.loads(...))
```

Step 13: check it by reading

Print the start of each Biden lead from 2020 on.

In which year's lead is he the 46th president, not about to become it? Does that match the model's year?

```
for row in leads[(leads["title"] == "Joe Biden")
                 & (leads["year"] >= "2020")].itertuples():
    print(row.id, "|", row.lead[:300], "\n")
```

Microsprint: a question with no answer

Ask the same about Kamala Harris as the 47th president.

No lead says it. Did the model answer `null`, or did it name a year?

Why null matters

An answer you can check needs its sources, or an honest `null`.

Step 14: a tool list

A model can't fetch data. It can only write a request. An **MCP server** tells a program which requests it may write.

This given cell loads the Census Bureau's server's tool list, saved on November 9.

```
tools = requests.get(cmcp + "tools.json",
                     headers=HEADERS).json()
for tool in tools:
    print(tool["name"], tool["inputSchema"]["required"])
```

Step 14: offer the model the tools

A given cell puts each tool in OpenRouter's form, `functions`, and sets week 11's question.

Send them together. `tool_choice` `"auto"` lets the model pick a tool.

```
body = {"models": MODELS, "temperature": 0,
        "tools": functions, "tool_choice": "auto",
        "messages": [
            {"role": "system",
             "content": TOOL_INSTRUCTION},
            {"role": "user", "content": question}]}
```

Step 14: the model writes one tool call

The model answers with a tool call: a tool's name and its arguments.

Which tool did it pick, and with which arguments?

```
response = requests.post(...).json()
message = response["choices"][0]["message"]
call = message["tool_calls"][0]["function"]
print(response["model"], call["name"], call["arguments"])
```

Step 14: turn the call into week 11's request

The model only wrote the call. Our code runs it. This given cell checks the tool's name, then builds week 11's address from the arguments.

```
args = json.loads(call["arguments"])
print(call["name"], call["name"] == "fetch-aggregate-data")
url = ("https://api.census.gov/data/"
       f"{args['year']}/{args['dataset']}")
if "group" in args["get"]:
    get = f"group({args['get']['group']})"
else:
    get = ",".join(args["get"]["variables"])
```

Step 14: run it

Send it with your Census key. Print the data, never the URL: your key is in it.

Is the median income the same as week 11's?

```
params = {"get": get, "for": args["for"],
          "in": args["in"], "key": census_key}
data = requests.get(url, params=params,
                    headers=HEADERS).json()
print(data)
```

What your code still decides

Which tools run. A model can pick any tool it is shown. Our code runs only one.

Which arguments go out. They decide what your code asks for, with your key.

Whether the answer is right. We know it because week 11 found it.

Microsprint

Ask for the city of Boulder by name, with no FIPS codes. Which tool does the model pick? Can our code run it?

Step 14: a skill is a Markdown file

A **skill** is a file of instructions for a model. Its front matter gives a name and a description, which tell a model when to read the rest.

Which of week 9's tools does this one teach?

```
skill = requests.get("https://raw.githubusercontent.com/"
    "anthropics/skills/main/skills/pdf/SKILL.md",
    headers=HEADERS).text
print("\n".join(skill.splitlines()[:5]))
print([line for line in skill.splitlines()
       if line.startswith("### ")])
```

Step 15: what did we learn?

Answer in the notebook, as comments.

- Which of Datalab's output would you publish?
- Which reports did the model get wrong, and why?
- Which jump could you explain only by reading?
- What will you check your project's model outputs against?

Due Sunday, Nov 22

Chapter 13's Recommended Exercises, from its companion notebook, on a public PDF you choose. Graded on a good-faith attempt.

Key takeaways

1. Call a model **as a function**: one request in, JSON or one tool call out. Your code decides what runs.
2. **Valid JSON can still be wrong.** Check it against something you trust.
3. Datalab **read** some of its output and its model **wrote** the rest.
4. The same request can return a different answer. **Record** the model, the date, and the response.
5. A similarity says how much a text changed. **Read** it to learn why.